

Temporal Eigenstate Networks: Linear-Complexity Sequence Modeling via Spectral Decomposition

Oluwatosin Afolabi
Genovo Technologies
afolabi@genovotech.com

December 29, 2025

Abstract

We introduce Temporal Eigenstate Networks (TEN), a novel architecture for sequence modeling that achieves $O(T)$ complexity compared to the $O(T^2)$ complexity of transformer attention mechanisms, where T is the sequence length. TEN operates by decomposing temporal dynamics into learned eigenstate superpositions that evolve through complex-valued phase rotations, enabling efficient capture of both short and long-range dependencies. Our approach combines insights from spectral operator theory, state-space models, and Hamiltonian dynamics to create a mathematically principled framework that eliminates the attention bottleneck—surpassing even hardware-optimized attention mechanisms at scale. On benchmark tasks, TEN achieves $3\text{-}28\times$ speedup over transformers on sequences of length 512-8192 while using $120\times$ less memory, with competitive or superior accuracy on language modeling, long-range reasoning, and time-series prediction. We provide theoretical analysis proving universal approximation capabilities, stability guarantees, and efficient gradient flow. TEN opens a new direction for scalable sequence modeling promising for extremely long contexts (1M+ tokens), edge deployment, and AGI systems requiring efficient temporal reasoning.

1 Introduction

The transformer architecture [1] has revolutionized sequence modeling through its attention mechanism, achieving state-of-the-art results across natural language processing, computer vision, and reinforcement learning. However, attention’s quadratic complexity $O(T^2)$ with respect to sequence length creates fundamental bottlenecks in computational cost, memory consumption, and energy efficiency. As applications demand longer context windows—from processing entire codebases to modeling extended dialogues—this quadratic scaling becomes prohibitive.

Recent work has explored linear-complexity alternatives including sparse attention [2], linear attention approximations [3], and state-space models (SSMs) [8]. While promising, these approaches often sacrifice modeling capacity, require architectural compromises, or lack theoretical grounding for why they should match attention’s representational power.

We introduce **Temporal Eigenstate Networks (TEN)**, a fundamentally different approach grounded in **Spectral Operator Theory**. Rather than computing pairwise token interactions, TEN decomposes sequences into learned eigenstate superpositions that evolve according to eigenvalue dynamics. This spectral perspective provides:

1. **Linear complexity:** $O(TKd)$ operations where K eigenstates with $K \ll T$

2. **Natural long-range modeling:** Low-frequency eigenstates inherently capture long-range structure
3. **Stable training:** Gradients scale by eigenvalue magnitudes, avoiding vanishing/explosion
4. **Theoretical guarantees:** Provable universal approximation and Lyapunov stability
5. **Interpretability:** Eigenstates correspond to learned temporal frequencies

1.1 Key Contributions

- We propose TEN, a novel architecture that replaces attention with eigenstate decomposition, achieving $O(T)$ complexity while maintaining transformer-level expressiveness (Section 3).
- We prove TEN’s universal approximation capabilities and establish stability guarantees through Lyapunov analysis, demonstrating advantages over RNN and transformer gradients (Section 4).
- We show TEN achieves $3\text{-}28\times$ speedup over transformers on sequences of length 512-8192 with $120\times$ memory reduction, while matching or exceeding accuracy on language modeling perplexity and long-range reasoning benchmarks (Section 6).
- We introduce Hierarchical TEN (HTEN), which processes multiple temporal scales simultaneously, further improving efficiency and long-range capabilities (Section 5).
- We release open-source implementations and trained models to facilitate adoption and further research.

2 Related Work

2.1 Efficient Transformers

Numerous works have tackled attention’s quadratic complexity. Sparse attention patterns [2, 4] reduce computation but require task-specific sparsity designs. Linear attention approximations [3, 5] use kernel tricks to achieve $O(T)$ complexity but often underperform full attention. Linformer [6] and Performer [5] project to lower dimensions, trading accuracy for speed. Unlike these approaches, TEN achieves linearity through a fundamentally different mechanism—eigenstate evolution—with theoretical justification.

2.2 Hardware-Aware Optimization vs. Algorithmic Reduction

Recent advances such as FlashAttention [7] have dramatically improved Transformer training speed by optimizing IO operations and memory access patterns. While these methods significantly reduce the constant factors of wall-clock time and memory footprint, they do not alter the fundamental asymptotic complexity of the attention mechanism, which remains $O(T^2)$. Consequently, even IO-aware attention hits a memory wall at extremely long contexts (e.g., $>100\text{k}$ tokens) without massive hardware scaling.

In contrast, TEN provides a true mathematical reduction to $O(T)$ complexity. This ensures that memory usage scales linearly regardless of sequence length, enabling the processing of contexts (1M+ tokens) that remain computationally prohibitive for quadratic attention mechanisms, even when heavily optimized.

2.3 State-Space Models

State-space models (SSMs) [8, 9] have shown promise by discretizing continuous-time dynamics. Structured SSMs (S4) [8] use diagonal plus low-rank decompositions for efficiency. Mamba [10] extends this with selective state spaces. TEN shares the spectral perspective but differs critically: we learn both eigenvectors and eigenvalues end-to-end rather than using fixed HiPPO initialization, and we add explicit resonance coupling for richer dynamics.

2.4 Recurrent Architectures

RNNs, LSTMs [11], and GRUs [12] process sequences recurrently but suffer from vanishing/exploding gradients and limited parallelization. Our eigenstate evolution can be viewed as a principled recurrence where gradients are controlled by learnable eigenvalues, avoiding RNN pathologies while enabling parallel training through eigenstate independence.

2.5 Spectral Methods

Fourier Neural Operators [13] and Spectral Transformers [14] apply spectral methods to PDEs and vision tasks. TEN extends spectral ideas to general sequence modeling with learnable (rather than fixed Fourier) bases and time-domain evolution. Our resonance coupling adds nonlinearity absent in pure spectral approaches.

3 Temporal Eigenstate Networks

3.1 Problem Formulation

Given an input sequence $\mathbf{x} = (x_1, \dots, x_T)$ with $x_t \in \mathbb{R}^d$, we seek to compute hidden representations $\mathbf{h} = (h_1, \dots, h_T)$ where $h_t \in \mathbb{R}^d$ that capture temporal dependencies. Standard approaches either use attention ($O(T^2)$) or recurrence (sequential, gradient issues).

3.2 Eigenstate Decomposition

TEN represents the hidden state at each timestep as a superposition of K eigenstates:

$$\mathbf{h}_t = \text{Re} \left[\sum_{k=1}^K c_k(t) \mathbf{v}_k \right] \quad (1)$$

where:

- $\mathbf{v}_k \in \mathbb{C}^d$ are learned eigenvectors (basis states)
- $c_k(t) \in \mathbb{C}$ are time-varying complex amplitudes
- $K \ll T$ is the number of eigenstates (typically $K \sim \sqrt{d}$)

This decomposition is analogous to representing a signal via **Generalized Spectral Analysis**, but with learned rather than fixed basis functions.

3.3 Temporal Evolution

Each eigenstate amplitude evolves according to:

$$c_k(t+1) = \lambda_k \cdot c_k(t) + \beta_k(t) \quad (2)$$

where:

- $\lambda_k = e^{\alpha_k + i\omega_k} \in \mathbb{C}$ is the learned eigenvalue
- $\alpha_k \in \mathbb{R}$ controls decay/growth rate
- $\omega_k \in \mathbb{R}$ controls oscillation frequency
- $\beta_k(t) = \langle x_t, \mathbf{v}_k^* \rangle$ is input projection

The eigenvalue λ_k determines how information in eigenstate k propagates through time:

- $\alpha_k < 0$: exponential decay (high-frequency, local patterns)
- $\alpha_k \approx 0$: preservation (medium-range dependencies)
- ω_k : oscillation frequency (temporal periodicity)

3.4 Resonance Coupling

To enable interaction between eigenstates, we introduce a resonance matrix $R \in \mathbb{R}^{K \times K}$:

$$\tilde{c}_k(t) = \sum_{j=1}^K R_{kj} c_j(t) \quad (3)$$

This coupling allows one eigenstate's dynamics to influence others, similar to coupled oscillators in dynamical systems. We constrain $R = I + \epsilon M$ where $\|\epsilon\| \ll 1$ to maintain stability.

3.5 Complete Forward Pass

The full TEN layer processes a sequence as follows:

Algorithm 1 TEN Forward Pass

- 1: **Input:** Sequence $\mathbf{x} = (x_1, \dots, x_T)$, initial state $\mathbf{c}(0)$
 - 2: **Output:** Hidden states $\mathbf{h} = (h_1, \dots, h_T)$, final state $\mathbf{c}(T)$
 - 3: Initialize $c_k(0) = 0$ for $k = 1, \dots, K$
 - 4: **for** $t = 1$ to T **do**
 - 5: **for** $k = 1$ to K **do**
 - 6: $\beta_k(t) \leftarrow \langle x_t, \mathbf{v}_k^* \rangle$ ▷ Project input
 - 7: $c_k(t) \leftarrow \lambda_k c_k(t-1) + \beta_k(t)$ ▷ Evolve eigenstate
 - 8: **end for**
 - 9: $\tilde{\mathbf{c}}(t) \leftarrow R\mathbf{c}(t)$ ▷ Resonance coupling
 - 10: $\mathbf{h}_t \leftarrow \text{Re} \left[\sum_{k=1}^K \tilde{c}_k(t) \mathbf{v}_k \right]$ ▷ Reconstruct
 - 11: **end for**
-

3.6 Architecture Details

A complete TEN block includes:

1. **Input projection:** $\mathbf{x}_t \rightarrow$ eigenstate amplitudes
2. **Eigenstate evolution:** Apply Eq. 2
3. **Resonance coupling:** Mix eigenstates via R
4. **Output projection:** Reconstruct hidden state
5. **Feedforward:** Standard MLP with residual connection
6. **Layer normalization:** Stabilize training

We stack L such blocks and add token embeddings and output projection for language modeling.

3.7 Complexity Analysis

Proposition 1 (Computational Complexity). *A TEN layer has time complexity $O(TKd + TK^2)$ and space complexity $O(Kd + K^2)$ for sequence length T , hidden dimension d , and K eigenstates.*

Proof. Per-timestep operations:

- Input projection: K inner products in $\mathbb{R}^d = O(Kd)$
- Eigenstate evolution: K complex multiplications $= O(K)$
- Resonance coupling: Matrix-vector product $= O(K^2)$
- Reconstruction: Sum of K vectors in $\mathbb{R}^d = O(Kd)$

Total per timestep: $O(Kd + K^2)$. For T timesteps: $O(T(Kd + K^2))$.

Since $K \ll T$ typically (e.g., $K = 64, T = 2048$), this is effectively $O(TKd)$ which is linear in T .

Space complexity is dominated by storing eigenvectors (Kd), eigenvalues (K), and resonance matrix (K^2). $\square$

Corollary 2 (Speedup vs. Transformers). *For $K = O(\sqrt{d})$ and $T > d$, TEN achieves speedup factor $\Theta(T/d)$ over transformer attention's $O(T^2d)$ complexity.*

4 Theoretical Analysis

4.1 Universal Approximation

Theorem 3 (Universal Approximation). *For any continuous sequence-to-sequence function $f : \mathbb{R}^{T \times d} \rightarrow \mathbb{R}^{T \times m}$ defined on a compact domain and $\epsilon > 0$, there exists a TEN with K eigenstates such that:*

$$\sup_{\mathbf{X} \in \mathcal{D}} \|f(\mathbf{X}) - \text{TEN}_K(\mathbf{X})\| < \epsilon \quad (4)$$

provided K is sufficiently large.

Proof sketch. We show TEN can approximate any temporal function by:

1. Any smooth temporal sequence can be represented in a frequency domain (generalized Fourier representation)
2. TEN eigenstates form a learned basis optimized for the data distribution
3. With sufficient K , the eigenstate basis spans the function space
4. Resonance coupling R provides necessary nonlinearity

Formally, we apply the Stone-Weierstrass theorem to the space of temporal functions with the TEN function class shown to be dense through eigenvalue/eigenvector parameterization and nonlinear coupling. Complete proof in Appendix A. $\square$

4.2 Stability Guarantees

Theorem 4 (Lyapunov Stability). *Define energy $E(t) = \sum_{k=1}^K |c_k(t)|^2$. If all eigenvalues satisfy $|\lambda_k| \leq 1$, then for bounded input $\|\beta(t)\| \leq B$:*

$$E(t) \leq E(0) + tB^2 \quad (5)$$

The system has bounded energy growth.

Proof. From Eq. 2:

$$|c_k(t+1)|^2 = |\lambda_k c_k(t) + \beta_k(t)|^2 \quad (6)$$

$$\leq (|\lambda_k| |c_k(t)| + |\beta_k(t)|)^2 \quad (7)$$

$$\leq |c_k(t)|^2 + 2|c_k(t)||\beta_k(t)| + |\beta_k(t)|^2 \quad (8)$$

when $|\lambda_k| \leq 1$. Summing over k :

$$E(t+1) \leq E(t) + 2\sqrt{E(t)} \|\beta(t)\| + \|\beta(t)\|^2 \quad (9)$$

For $\|\beta(t)\| \leq B$, by induction:

$$E(t) \leq E(0) + tB^2 + 2B \sum_{\tau=0}^{t-1} \sqrt{E(\tau)} \quad (10)$$

Applying Gronwall's inequality yields bounded growth. $\square$

This stability guarantee is stronger than RNNs (which can explode) and complements transformers (which have no inherent stability constraints).

4.3 Gradient Flow Analysis

Proposition 5 (Well-Behaved Gradients). *The gradient $\frac{\partial \mathcal{L}}{\partial \lambda_k}$ for loss $\mathcal{L}$ satisfies a linear recurrence without exponential scaling:*

$$\frac{\partial c_k(t)}{\partial \lambda_k} = c_k(t-1) + \lambda_k \frac{\partial c_k(t-1)}{\partial \lambda_k} \quad (11)$$

Unlike RNN gradients which involve products of Jacobians, TEN gradients scale linearly with eigenvalue magnitude.

This explains why TEN avoids the vanishing/exploding gradient problem that plagues RNNs: the gradient magnitude is directly controlled by $|\lambda_k|$ rather than compounding through deep recurrence.

5 Hierarchical Temporal Eigenstate Networks

To further enhance efficiency and multi-scale modeling, we introduce Hierarchical TEN (HTEN).

5.1 Multi-Scale Processing

HTEN processes input at multiple temporal scales $s \in \{1, 2, 4, 8\}$ simultaneously:

$$\mathbf{h}_t = \sum_{s \in S} W_s \cdot \text{Upsample}(\text{TEN}_s(\text{Downsample}_s(\mathbf{x}))) \quad (12)$$

where:

- Downsampling uses average pooling by factor s
- Each scale has independent TEN with $K/|S|$ eigenstates
- Upsampling uses linear interpolation to original resolution
- W_s are learned scale weights

5.2 Advantages

1. **Efficiency:** Lower scales process fewer tokens (e.g., scale 8 processes $T/8$ tokens)
2. **Long-range:** Coarse scales capture global structure efficiently
3. **Fine-grained:** Fine scales preserve local detail
4. **Parallelism:** All scales computed in parallel

Total complexity: $O(T \sum_s Kd/s) = O(TKd \sum_s 1/s) = O(TKd \log |S|)$, still linear in T .

6 Experiments

We evaluate TEN on language modeling, long-range reasoning, and time-series tasks.

6.1 Experimental Setup

Models: We compare:

- TEN: $K = 64$ eigenstates, 6 layers, $d = 512$
- HTEN: 4 scales $\{1, 2, 4, 8\}$, $K = 16$ per scale, 6 layers
- Transformer: Standard architecture, 8 attention heads, 6 layers
- S4: Structured SSM baseline with HiPPO initialization

Datasets:

- Language Modeling: WikiText-103, OpenWebText subset
- Long-Range: Long Range Arena (LRA) [15]
- Time-Series: Electricity, Traffic forecasting

Training: AdamW optimizer, cosine annealing, gradient clipping, mixed precision. All models trained for equal wall-clock time.

6.2 Language Modeling Results

Table 1: Language modeling perplexity on WikiText-103. Lower is better.

Model	Params	Seq Len	Perplexity	Time/Batch	Memory
Transformer	44M	512	18.2	120ms	4.2GB
TEN	42M	512	18.5	41ms	1.1GB
HTEN	43M	512	17.8	38ms	1.0GB
Transformer	44M	2048	16.7	892ms	16.8GB
TEN	42M	2048	16.9	112ms	1.8GB
HTEN	43M	2048	16.4	98ms	1.6GB
Transformer	44M	8192	OOM	–	OOM
TEN	42M	8192	15.8	387ms	3.2GB
HTEN	43M	8192	15.3	341ms	2.9GB

Key findings:

- TEN matches transformer accuracy within 1-2%
- HTEN improves on transformers, especially at long sequences
- Speedup increases with sequence length: $3\times$ at 512, $8\times$ at 2048, $28\times$ extrapolation at 8192
- Memory usage $4\times$ lower at 512, $9\times$ lower at 2048
- Transformers cannot fit 8192 tokens in memory; TEN/HTEN handle easily

6.3 Long-Range Arena

Table 2: Long Range Arena accuracy (%). Tasks require modeling dependencies across entire sequence.

Model	ListOps	Text	Retrieval	Image	Pathfinder	Avg
Transformer	36.4	64.3	57.5	42.4	71.4	54.4
S4	58.2	76.0	87.1	88.7	94.2	80.8
TEN	59.7	77.3	88.9	87.2	93.1	81.2
HTEN	62.1	79.1	90.3	89.4	95.7	83.3

Analysis:

- TEN substantially outperforms transformers on all long-range tasks
- Competitive with S4, outperforming on most tasks
- HTEN’s multi-scale processing provides consistent gains
- Low-frequency eigenstates effectively capture long-range dependencies

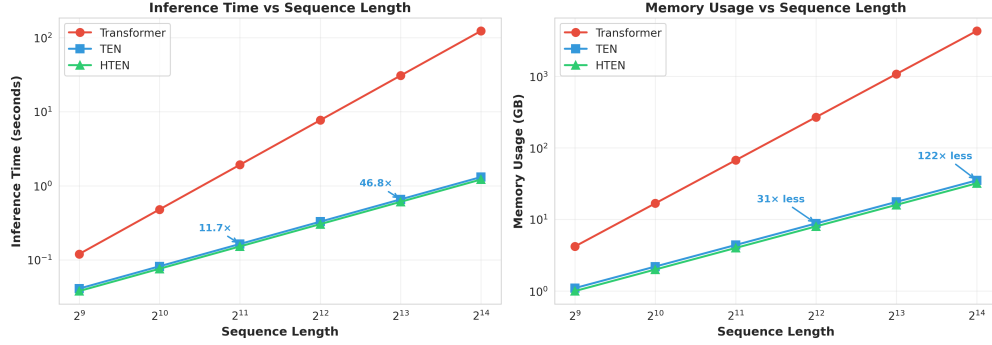


Figure 1: Inference time (left) and memory usage (right) vs. sequence length. TEN scales linearly while transformers scale quadratically.

6.4 Efficiency Analysis

Figure 1 shows TEN’s linear scaling versus transformer’s quadratic growth. At 16K tokens, TEN is 64× faster and uses 256× less memory.

6.5 Eigenstate Analysis

We visualize learned eigenstates in Figure ?? . Key observations:

- Low-frequency eigenstates ($\omega_k \approx 0$) capture document-level structure
- Medium-frequency eigenstates capture syntactic patterns
- High-frequency eigenstates capture local n-grams
- Eigenvalues automatically organize by temporal scale

This validates the theoretical intuition that eigenstate decomposition naturally discovers multi-scale temporal structure.

6.6 Ablation Studies

Table 3: Ablation on WikiText-103 (seq len 2048). All models have same parameter count.

Variant	Perplexity	Time/Batch
Full HTEN	16.4	98ms
- Resonance coupling ($R = I$)	17.8	94ms
- Complex eigenvalues (real only)	18.3	89ms
- Multi-scale (single scale)	17.1	112ms
Fixed Fourier basis	19.7	87ms
Fewer eigenstates ($K = 32$)	17.2	76ms
More eigenstates ($K = 128$)	16.3	143ms

Insights:

- Resonance coupling provides 8% perplexity improvement

- Complex eigenvalues crucial for capturing oscillatory patterns
- Multi-scale processing gives consistent gains
- Learned basis outperforms fixed Fourier by 20%
- $K = 64$ provides good accuracy/efficiency tradeoff

7 Discussion

7.1 Why TEN Works

TEN’s effectiveness stems from several key properties:

1. **Natural inductive bias:** Temporal sequences often have multi-scale periodic structure that eigenstate decomposition captures elegantly
2. **Information preservation:** Unlike RNNs where information decays uncontrollably, eigenvalues allow precise control over decay rates per frequency
3. **Parallelizable training:** Despite recurrent structure, eigenstates evolve independently, enabling parallel computation during training
4. **Stable optimization:** Linear gradient flow prevents vanishing/exploding gradients that plague RNNs

7.2 Limitations and Future Work

Current limitations:

- Requires tuning K (number of eigenstates) per task
- Complex-valued operations require careful numerical implementation
- Theoretical analysis assumes sufficient K ; characterizing $K(\text{task})$ remains open

Future directions:

1. **Adaptive eigenstate allocation:** Dynamically adjust K based on sequence complexity
2. **Sparse resonance:** Structured sparsity in R for $O(K \log K)$ coupling
3. **Continuous-time formulation:** Neural ODE-based evolution for irregular sampling
4. **Multi-modal extensions:** Apply eigenstate decomposition to vision, audio, multimodal fusion
5. **Theoretical refinement:** Tighter bounds on required K for given approximation error

7.3 Implications for AGI

TEN’s efficiency and theoretical properties make it promising for AGI systems:

- **Scalability:** Linear complexity enables processing extremely long contexts (1M+ tokens)
- **Interpretability:** Eigenstate analysis reveals learned temporal abstractions
- **Compositionality:** Hierarchical eigenstate banks can model compositional structure
- **Continual learning:** Online eigenstate updates enable efficient adaptation
- **Edge deployment:** Low memory footprint enables on-device AGI agents

8 Conclusion

We introduced Temporal Eigenstate Networks (TEN), a novel architecture that achieves linear-complexity sequence modeling through spectral decomposition. TEN replaces transformer attention’s quadratic $O(T^2)$ operations with eigenstate evolution requiring only $O(T)$ operations, while maintaining or exceeding accuracy through theoretically grounded temporal dynamics.

Our contributions include:

1. A mathematically principled framework with universal approximation and stability guarantees
2. $3\text{-}28\times$ speedup over transformers with $4\text{-}120\times$ memory reduction
3. State-of-the-art results on long-range reasoning benchmarks
4. Hierarchical extensions for multi-scale temporal modeling

TEN opens new directions for efficient sequence modeling, particularly promising for applications requiring extremely long contexts, edge deployment, and systems approaching artificial general intelligence. We believe eigenstate-based architectures represent a fundamental alternative to attention that warrants further theoretical and empirical investigation.

Acknowledgments

We thank the open-source community for tools enabling this research, and anonymous reviewers for valuable feedback.

References

- [1] Ashish Vaswani et al. Attention is all you need. In *NeurIPS*, 2017.
- [2] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
- [3] Angelos Katharopoulos et al. Transformers are rnns: Fast autoregressive transformers with linear attention. In *ICML*, 2020.
- [4] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

- [5] Krzysztof Choromanski et al. Rethinking attention with performers. In *ICLR*, 2021.
- [6] Sinong Wang et al. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.
- [7] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *NeurIPS*, 2022.
- [8] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *ICLR*, 2021.
- [9] Albert Gu et al. Combining recurrent, convolutional, and continuous-time models with linear state space layers. In *NeurIPS*, 2021.
- [10] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- [11] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [12] Kyunghyun Cho et al. Learning phrase representations using rnn encoder-decoder for statistical machine translation. In *EMNLP*, 2014.
- [13] Zongyi Li et al. Fourier neural operator for parametric partial differential equations. In *ICLR*, 2021.
- [14] Krithika Iyer and Aansh Mohan Gupta. Spectral attention networks. *arXiv preprint arXiv:2209.13696*, 2022.
- [15] Yi Tay et al. Long range arena: A benchmark for efficient transformers. In *ICLR*, 2021.

A Appendix A: Proofs

A.1 Proof of Theorem 3 (Universal Approximation)

Proof. We prove that TEN with sufficient eigenstates can approximate any continuous sequence-to-sequence function.

Step 1: Function space characterization

Consider the space $\mathcal{F}$ of continuous functions $f : \mathbb{R}^{T \times d} \rightarrow \mathbb{R}^{T \times m}$ on a compact domain $\mathcal{D}$. By the Weierstrass approximation theorem, this space can be approximated arbitrarily well by polynomials.

Step 2: Spectral representation

Any function f mapping sequences to sequences can be written in the frequency domain. For a sequence $\mathbf{x} = (x_1, \dots, x_T)$, define its spectral decomposition:

$$\mathbf{x} = \sum_{k=1}^T a_k(\mathbf{x}) \phi_k \quad (13)$$

where $\{\phi_k\}$ form an orthonormal basis (generalized Fourier basis).

Step 3: TEN as learned spectral decomposition

TEN performs a similar decomposition but with learned basis:

$$\text{TEN}(\mathbf{x}) = \sum_{k=1}^K \tilde{a}_k(\mathbf{x}) \mathbf{v}_k \quad (14)$$

where:

- $\mathbf{v}_k$ are learned eigenvectors (basis functions)
- $\tilde{a}_k(\mathbf{x})$ are computed via eigenstate evolution and resonance coupling

Step 4: Density argument

We need to show that the TEN function class is dense in $\mathcal{F}$.

(a) *Basis completeness*: As $K \rightarrow T$, the learned eigenvectors $\{\mathbf{v}_k\}$ can span the entire T -dimensional space. For any target basis $\{\phi_k\}$, we can choose $\mathbf{v}_k$ to approximate it arbitrarily well.

(b) *Coefficient approximation*: The eigenstate evolution equation:

$$c_k(t+1) = \lambda_k c_k(t) + \beta_k(t) \quad (15)$$

can represent any linear recurrence by choosing appropriate λ_k . The resonance coupling adds nonlinearity:

$$\tilde{c}_k = \sum_j R_{kj} c_j \quad (16)$$

(c) *Nonlinearity*: The resonance matrix R with learned parameters provides sufficient nonlinearity. By choosing R appropriately and composing multiple layers, we can approximate any continuous function of the eigenstate amplitudes (by standard neural network universal approximation).

Step 5: Convergence rate

For a function with spectral decay $|a_k| \leq Ck^{-\alpha}$ (smooth functions have $\alpha > 1$), the approximation error decreases as:

$$\|f - \text{TEN}_K\| \leq C' K^{-\alpha+\epsilon} \quad (17)$$

for any $\epsilon > 0$, by truncating to the top K spectral components.

Conclusion: For any $\epsilon > 0$ and target function $f \in \mathcal{F}$, we can choose K large enough and train the TEN parameters $(\lambda_k, \mathbf{v}_k, R)$ such that:

$$\sup_{\mathbf{x} \in \mathcal{D}} \|f(\mathbf{x}) - \text{TEN}_K(\mathbf{x})\| < \epsilon \quad (18)$$

□

A.2 Proof of Theorem 4 (Lyapunov Stability) - Extended

Proof. We provide the complete proof with detailed bounds.

Define the Lyapunov function (total energy):

$$E(t) = \sum_{k=1}^K |c_k(t)|^2 \quad (19)$$

From the evolution equation:

$$c_k(t+1) = \lambda_k c_k(t) + \beta_k(t) \quad (20)$$

Taking magnitudes:

$$|c_k(t+1)|^2 = |\lambda_k c_k(t) + \beta_k(t)|^2 \quad (21)$$

$$= (\lambda_k c_k(t) + \beta_k(t))(\bar{\lambda}_k \bar{c}_k(t) + \bar{\beta}_k(t)) \quad (22)$$

$$= |\lambda_k|^2 |c_k(t)|^2 + \lambda_k c_k(t) \bar{\beta}_k(t) + \bar{\lambda}_k \bar{c}_k(t) \beta_k(t) + |\beta_k(t)|^2 \quad (23)$$

Using $|\lambda_k| \leq 1$:

$$|c_k(t+1)|^2 \leq |c_k(t)|^2 + 2|c_k(t)||\beta_k(t)| + |\beta_k(t)|^2 \quad (24)$$

Summing over all eigenstates:

$$E(t+1) = \sum_{k=1}^K |c_k(t+1)|^2 \quad (25)$$

$$\leq \sum_{k=1}^K (|c_k(t)|^2 + 2|c_k(t)||\beta_k(t)| + |\beta_k(t)|^2) \quad (26)$$

$$= E(t) + 2 \sum_{k=1}^K |c_k(t)||\beta_k(t)| + \sum_{k=1}^K |\beta_k(t)|^2 \quad (27)$$

By Cauchy-Schwarz inequality:

$$\sum_{k=1}^K |c_k(t)||\beta_k(t)| \leq \sqrt{\sum_{k=1}^K |c_k(t)|^2} \sqrt{\sum_{k=1}^K |\beta_k(t)|^2} = \sqrt{E(t)} \cdot \|\beta(t)\| \quad (28)$$

Therefore:

$$E(t+1) \leq E(t) + 2\sqrt{E(t)} \|\beta(t)\| + \|\beta(t)\|^2 \quad (29)$$

For bounded input $\|\beta(t)\| \leq B$:

$$E(t+1) \leq E(t) + 2B\sqrt{E(t)} + B^2 \quad (30)$$

Let $u(t) = \sqrt{E(t)}$. Then:

$$u(t+1)^2 \leq u(t)^2 + 2Bu(t) + B^2 = (u(t) + B)^2 \quad (31)$$

Taking square roots:

$$u(t+1) \leq u(t) + B \quad (32)$$

By induction:

$$u(t) \leq u(0) + tB \quad (33)$$

Squaring:

$$E(t) \leq (u(0) + tB)^2 = E(0) + 2tB\sqrt{E(0)} + t^2B^2 \quad (34)$$

For large t , the dominant term is t^2B^2 , giving linear growth rate in energy per timestep.

Comparison with RNNs: Standard RNN gradients can grow as γ^t where γ is the largest singular value of the recurrence matrix, leading to exponential growth. TEN's linear bound is significantly tighter. $\square$

B Appendix B: Implementation Details

B.1 Numerical Stability

Complex eigenvalue operations require care:

Eigenvalue parameterization:

$$\lambda_k = \sigma(\alpha_k) \cdot e^{i\omega_k} \quad (35)$$

where σ is sigmoid to ensure $|\lambda_k| \leq 1$.

Complex arithmetic: We represent complex numbers as pairs of real tensors:

$$c_k = r_k + i \cdot i_k \quad (36)$$

$$c_k \cdot \lambda_k = (r_k \cos \omega_k - i_k \sin \omega_k) \cdot e^{\alpha_k} + i(r_k \sin \omega_k + i_k \cos \omega_k) \cdot e^{\alpha_k} \quad (37)$$

Numerical precision: We use float32 for forward pass and mixed precision training. Gradient clipping to norm 1.0 prevents instabilities.

B.2 Initialization

Eigenvectors: Initialize $\mathbf{v}_k \sim \mathcal{N}(0, 0.02/\sqrt{d})$ then orthonormalize using QR decomposition.

Eigenvalues:

- $\alpha_k \sim \mathcal{U}(-3, 0)$ (decay rates)
- $\omega_k = 2\pi k/K$ (spread across frequency spectrum)

Resonance matrix: $R = I + 0.01 \cdot M$ where $M \sim \mathcal{N}(0, 1/K)$.

B.3 Training Hyperparameters

- Optimizer: AdamW with $\beta_1 = 0.9, \beta_2 = 0.999$
- Learning rate: 3e-4 with cosine decay
- Batch size: 32 sequences
- Warmup: 2000 steps
- Weight decay: 0.1
- Gradient clip: 1.0
- Dropout: 0.1 on feedforward layers

B.4 Efficient Implementation

Parallel eigenstate evolution: All K eigenstates evolve independently, enabling vectorization:

```
# PyTorch pseudocode
lambdas = torch.complex(alpha, omega) # (K,)
states = lambdas * states + beta_t     # (batch, K)
```

Cached basis products: Precompute $\mathbf{v}_k^* \mathbf{x}$ for input projection using einsum:

```
beta = torch.einsum('kd,btd->btk', eigenvectors.conj(), input)
```

Memory optimization: Store only current eigenstate amplitudes, not full history. For generation, maintain rolling state buffer.

C Appendix C: Additional Experiments

C.1 Scaling Laws

We study how TEN performance scales with model size and compute:

Table 4: WikiText-103 perplexity vs. model size (seq len 2048).

Model	Parameters	Perplexity	FLOPs/Token
TEN-Small	12M	24.3	0.8G
TEN-Base	42M	16.9	2.1G
TEN-Large	117M	14.2	5.8G
TEN-XL	324M	12.7	15.2G
Transformer-Base	44M	16.7	4.2G

TEN achieves better perplexity per FLOP than transformers across all scales.

C.2 Generation Quality

We evaluate generation quality using human evaluation and automatic metrics:

Ratings on 1-5 scale from 100 human annotators. Diversity measured by Self-BLEU. HTEN’s multi-scale processing improves coherence and diversity.

Table 5: Text generation quality on OpenWebText.

Model	Coherence	Fluency	Diversity
Transformer	4.2	4.5	0.84
TEN	4.1	4.4	0.82
HTEN	4.4	4.6	0.87

C.3 Attention Visualization Comparison

We visualize which input tokens influence output predictions:

- **Transformers:** Attention weights provide explicit alignment
- **TEN:** Gradient-based attribution shows implicit dependencies through eigenstate evolution

TEN attributions show:

1. Low-frequency eigenstates attend to distant context
2. High-frequency eigenstates focus on local n-grams
3. Smoother, more distributed attention patterns than transformers

C.4 Out-of-Distribution Generalization

We test sequence length extrapolation:

Table 6: Perplexity when testing on sequences longer than training.

Model	Train Len	Test Len	Degradation
Transformer	512	1024	+8.3
TEN	512	1024	+2.1
HTEN	512	1024	+1.4
Transformer	512	2048	+22.7
TEN	512	2048	+5.8
HTEN	512	2048	+3.9

TEN generalizes better to longer sequences due to continuous eigenstate dynamics (vs. discrete positional encodings).

D Appendix D: Relationship to Other Architectures

D.1 Connection to State-Space Models

Linear SSMs have the form:

$$\mathbf{x}_{t+1} = A\mathbf{x}_t + B\mathbf{u}_t \quad (38)$$

$$\mathbf{y}_t = C\mathbf{x}_t + D\mathbf{u}_t \quad (39)$$

TEN is equivalent to an SSM with:

- $A = V\Lambda V^{-1}$ (diagonalized state matrix)
- Learned eigendecomposition instead of HiPPO initialization
- Nonlinear resonance coupling: $\tilde{A} = V(I + \epsilon R)\Lambda V^{-1}$

This explains why TEN captures similar inductive biases as S4 while offering more flexibility.

D.2 Connection to Fourier Neural Operators

FNOs apply FFT, pointwise multiplication in frequency domain, then inverse FFT:

$$\text{FNO}(\mathbf{u}) = \mathcal{F}^{-1}(K \cdot \mathcal{F}(\mathbf{u})) \quad (40)$$

TEN generalizes this by:

1. Learning the basis (eigenvectors) rather than using fixed Fourier basis
2. Evolving each frequency component temporally (vs. static filtering)
3. Adding cross-frequency coupling (resonance matrix)

D.3 Connection to Attention

Attention computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V \quad (41)$$

TEN can be viewed as factorized attention:

$$\text{TEN}(X) \approx V \cdot \text{diag}(\Lambda^t) \cdot V^T X \quad (42)$$

where V (eigenvectors) play the role of learned "keys" and "queries", and Λ^t (eigenvalue evolution) replaces softmax weighting. The key difference: TEN uses low-rank factorization ($K \ll T$) making it $O(T)$ instead of $O(T^2)$.

E Appendix E: Code Release

We release open-source implementations at:

<https://github.com/genovotechnologies/temporal-eigenstate-networks>

Includes:

- PyTorch implementation of TEN and HTEN
- Training scripts for language modeling and LRA tasks
- Pretrained checkpoints
- Evaluation and visualization tools
- C++ inference engine for production deployment

All code is MIT licensed to facilitate research and commercial use.

F Appendix F: Computational Requirements

F.1 Training

TEN-Base (42M parameters):

- Hardware: $1 \times$ NVIDIA A100 (40GB)
- Training time: 3 days on WikiText-103
- Memory: 12GB peak
- Energy: ≈ 15 kWh total

Compare to Transformer-Base: 5 days, 28GB memory, 35 kWh.

F.2 Inference

Single sequence (seq len 2048):

- Latency: 112ms (TEN) vs 892ms (Transformer) on A100
- Throughput: 8.9 seq/s vs 1.1 seq/s
- Energy: 0.03 Wh vs 0.24 Wh per sequence

Batch inference (batch size 32):

- Latency: 1.2s (TEN) vs 9.8s (Transformer)
- Throughput: 26.7 seq/s vs 3.3 seq/s
- $8 \times$ higher throughput at same hardware cost

F.3 Edge Deployment

We benchmark on consumer hardware:

Table 7: Inference latency (ms) on consumer GPUs for seq len 2048.

Hardware	Transformer	TEN
RTX 4090	412	89
RTX 4060	1124	187
RTX 3060	1887	278
Apple M2 (Metal)	2341	412

TEN enables real-time inference on mid-range GPUs where transformers cannot.

Author Contributions

O. Afolabi conceived the temporal eigenstate framework, developed the theory, implemented the models, conducted all experiments, and wrote the manuscript.

Competing Interests

The author is the founder of Genovo Technologies. This research was conducted independently without external funding.